

Cryptography, Theory and Practice CIS 4930/6930 (Fall 2026)

Instructor: Dr. Attila A. Yavuz

TA: Sean Hernandez

Contributor: Sheng Rao

September 1, 2026

Homework 1 (**Due: Sept. 11th, 2026 – 11 : 59**)[100 Points]

(Upload your solutions to the CANVAS)

Requirements: HW groups are 2 people. Each student should contribute all parts of the answers. If one group member drops, the other is responsible (each HW can be handled by a single student if needed, so no excuse for missing a partner. See syllabus for breaking-up with a programming partner). Directly borrowing (e.g., copy-paste) from any material and putting it in your solutions (e.g, from online solutions, Wikipedia, or research papers) is plagiarism (see Syllabus for its corresponding actions). Please cite very carefully each resource you use but citing a solution does not give a license to directly put it as an answer. **All of your answers must be in your own words and interpretations.**

Note: Many of these questions require you to recall explanations/discussions during the class (this is why attendance is important), and also do research on the internet to find good answers. Please kindly note that very brief and/or informal explanations will potentially not be sufficient.

Remark: Answers to questions should be prepared by a text editor (e.g., Microsoft Word or Latex). Handwritten submissions are not accepted. In your answer key, please indicate the point of each question as it is done in this HW document. Preferably, take questions from here, plug them into your answer key with their sub scores, and provide your answer below them. This will facilitate the grading.

As indicated in Syllabus, **no partial credit is possible for coding answers that does not produce correct results with respect to test vectors, deviate from the description and rubric, are coded on platforms other than specified, or do not work on the grader's machine.**

1 Question and Answer [25 pt]

1.1 One-time Pad Basics [5 pt]

- A. The One Time Pad (OTP) encryption is highly secure, but why do we not see it much in practice? Explain at least two reasons (some discussed in class!) [3 pt]
- B. Why does an ideal OTP encryption achieve information-theoretical security? [2 pt]

1.2 Security of cryptographic hash function: [10 pt]

What is the birthday attack against hash functions?

Given an n -bit hash output, mathematically formulate the upper bound of collision probability of it. Specifically, write how the birthday paradox is formulated. You can use approximations if needed.

1.3 Kerckhoffs's principle: [5 pt]

What is Kerckhoffs's principle in modern cryptography?

Write 2-3 paragraphs of explanation and why it is important.

1.4 Kerberos and Needham-Schroeder: [5 pt]

Describe the Kerberos and Needham-Schroeder protocols, specifically touching on how the two are related and how they function as challenge-response protocols.

Write 2-3 paragraphs of explanation.

- You Must submit 1 file for this part of the assignment:
[HW1Group\(YourGroupNumber\).pdf](#)
-

2 Programming Assignment 1: HMAC [30 pt]

In this exercise we will put into practice the basic cryptographic tools and the HMAC implementation that were discussed during the class. We will be putting into action the usage of Hashing to ensure authentication and integrity of messages. Specifically, consider the HMAC implementation as described in “CryptoHashFunctions.ppt”, slide 16-17, where a padded shared secret key is pre-pended to a message, hashed, then that hash result has the same secret key (but differently padded) prepended to that, then the entire thing hashed again. We will utilize the Blake 2s function from OpenSSL libraries as our hashing function. Moreover, as alluded to in Slide 17, the shared secret key will be processed to fit exactly the block size, then XORed with two different padding constants to get the inner and outer padded keys.

HMAC:

1. The program reads the message from the “**Message.txt**” file and shared secret key from the “**Shared-Key.txt**” file.
(Read them as unsigned char so that you can use the functions provided in in-class exercises).
2. The program converts the key to Hex format and writes that in a file named “**Key.txt**”.
3. The program processes the secret key so that it is exactly the block size of the hashing function by Hashing it if it is larger than the block size, then if it is smaller (guaranteed to be so if it was hashed) padding the right with zeroes until it is exactly the same size as the block size.
4. The program converts the processed key to Hex format and writes that in file named “**ProcessedKey.txt**”.
5. The program computes the HMAC using concatenation of the key (XORed with inner and outer padding constants) with the message, doing two different hashes as seen in the following equation:
$$HMAC \leftarrow H((k \oplus opad) || H((k \oplus ipad) || m)).$$
6. The program converts the HMAC to Hex format and writes it in a file named “**FinalHash.txt**”.

Programming Assignment 1 - Specific Notes

- Since we are using Blake 2s as the hashing function, the block size is 64 (in units of bytes, or 512 bits) and the hash result size is 32 (in units of bytes, or 256 bits). You can refer to the OpenSSL library documentation https://www.openssl.org/docs/manmaster/man3/EVP_DigestInit/ to learn how to use the OpenSSL digest (hashing) functions.

- The inner padding constant is the byte 0x36 repeated (Block size) times, the outer padding constant is the byte 0x5c repeated (Block size) times.
- In grading at our side, the message, shared key, and lengths will be chosen at random and everything must work.
- Compile your code with the following commands:
gcc hmac.c -lssl -lcrypto -o hmac
- Run your code for the first test files with the following commands:
./hmac Message1.txt SharedKey1.txt
- We have provided a script (VerifyingHMAC.sh) and test files that you can use to test the correctness of your final code (hmac.c). Please, only submit your code if it is functional.
- You can check your answer with each provided test file manually or you can use the script to verify your solution for the HMAC programming assignment.
- In order to use the script, just put “hmac.c” and the rest of the provided files along with the “VerifyingHMAC.sh” in one folder and run the following command in the terminal:
bash VerifyingHMAC.sh
- The verification script is the same one that is run to grade your submissions, only the test vectors are different.
- **You Must submit 1 file for this part of the assignment:**
hmac.c

Rubric:

- The “**Key.txt**” file **[10 pt]**
- The “**ProcessedKey.txt**” file **[10 pt]**
- The “**FinalHash.txt**” file **[10 pt]**

3 Programming Assignment 2: Challenge-Response Protocol **[45 pt]**

In this exercise we will put into action a Challenge-Response protocol similar to the ones that were discussed during the class. Specifically, consider how storing and using a counter and nonce as state information helps secure the authenticity and integrity of the message as described in “CryptoHashFunctionsandApplicationsFall2026.pptx”.

In this toy Challenge-Response implementation, Alice will have a message and shared secret key, as well as storing an incrementing counter and nonce as state information. Alice will encrypt the message by XORing it with the Hash of the concatenation of the secret key and counter, then sign the encrypted message (ciphertext) by using a keyed HMAC on the concatenation of the ciphertext and the nonce. Alice will send the ciphertext and signature to Bob as her Challenge. Bob, at this point, has the ciphertext, signature, shared secret key, and stores the same counter and nonce as Alice as his state information. Bob checks Alice’s signature to see if it matches, as he has the shared secret key, ciphertext, and the same nonce state as Alice. If the comparison is successful, Bob continues by decrypting the ciphertext with XOR in much the same manner as it was encrypted, getting the message back in plaintext. Bob computes his Response using the decrypted message, his counter (incremented by 1), and his nonce (incremented by 1). Bob sends this Response back to Alice, and increments his counter and nonce states for the next challenge/response with Alice. Alice receives Bob’s Response and compares this with the expected response, which is calculated the same as Bob’s, just with Alice’s counter and nonce. If the comparison is successful, Alice can be confident that Bob has received the accurate message, and Alice increments her counter and nonce states for the next challenge/response with Bob.

We will utilize SHA256 from the OpenSSL libraries as our hashing function and utilize the HMAC function from OpenSSL to get our keyed HMAC results.

Alice:

1. Alice reads the message, shared secret key, counter state, and nonce state from “**Message.txt**”, “**Shared-Key.txt**”, “**A_ctr.txt**”, and “**A_nonce.txt**” respectively.
(Read them as unsigned char so that you can use the functions provided in in-class exercises).
2. Alice writes the key in Hex format to a file named “**Key.txt**”.
3. Alice encrypts the message with XOR to attain the ciphertext: $c \leftarrow m \oplus H(k||ctr)$.
4. Alice writes the ciphertext in Hex format to a file named “**Ciphertext.txt**”.
5. Alice computes the signature using a keyed HMAC: $(sig \leftarrow HMAC_k(c||nonce))$.
6. Alice writes the signature in Hex format to a file named “**Signature.txt**”.
7. Once Bob has processed the message, Alice reads Bob’s response from the file named “**Response.txt**”.
8. Alice computes what the correct response should be from Bob ($response' \leftarrow H(m||(ctr+1)||(nonce+1))$) and compares it with Bob’s response (Note that Bob wrote the response in Hex format).
9. If the comparison is successful, Alice writes “Acknowledgment Successful” in a file called “**Acknowledgment.txt**.” Conversely, if the comparison fails, she records “Acknowledgment Failed.”
10. Alice increments and updates her counter and nonce files “**A_ctr.txt**” and “**A_nonce.txt**”.

Bob:

1. Bob reads the ciphertext, signature, shared secret key, his counter state, and his nonce state from “**Ciphertext.txt**”, “**Signature.txt**”, “**SharedKey.txt**”, “**B_ctr.txt**”, and “**B_nonce.txt**” respectively.
(Read them as unsigned char so that you can use the functions provided in in-class exercises).
2. Bob computes the expected signature using a keyed HMAC in the same manner as Alice ($(sig' \leftarrow HMAC_k(c||nonce))$) and checks it against the signature that Alice provided (Note that Alice wrote the signature in Hex format).
3. If the comparison is successful, Bob continues, otherwise the program exits.
4. Bob decrypts the ciphertext with XOR: $m \leftarrow c \oplus H(k||ctr)$.
5. Bob computes his response: $response \leftarrow H(m||(ctr+1)||(nonce+1))$.
6. Bob writes his response in Hex format to a file named “**Response.txt**”.
7. Bob increments and updates his counter and nonce files “**B_ctr.txt**” and “**B_nonce.txt**”.

Programming Assignment 2 - Specific Notes

- The Hash (SHA-256) and HMAC functions are called from the OpenSSL libraries. You can refer to the OpenSSL library documentation <https://www.openssl.org/docs/manmaster/man3/> to learn how to use these functions.
- The counter and nonce stored in the text files are decimal strings depicting integers in unsigned char form, it is intended to concatenate them as unsigned char but increment them as integers (so a nonce of the string “99” in unsigned char format is incremented to the string “100” in unsigned char format).

- Since Alice needs Bob's response to continue, if Bob's response does not exist yet then Alice must exit gracefully until the program is called again after Bob is run.
- In grading at our side, the seed, message, counter, nonce, and lengths will be chosen at will and everything must work. The message length, however, is guaranteed to be 256 bits or 32 bytes.
- Compile your codes with the following commands:
`gcc alice.c -lssl -lcrypto -o alice`
`gcc bob.c -lssl -lcrypto -o bob`
- Run your codes for the first test files with the following commands:
`./alice Message1.txt SharedKey1.txt A_ctr.txt A_nonce.txt`
`./bob Ciphertext.txt Signature.txt SharedKey1.txt B_ctr.txt B_nonce.txt`
- We have provided a script (VerifyingAliceBob.sh) and test files that you can use to test the correctness of your final codes (alice.c and bob.c). Please, only submit your code if it is functional.
- You can check your answer with each provided test file manually or you can use the script to verify your solution for this programming assignment.
- In order to use the script, just put "alice.c", "bob.c", and the rest of the provided files along with the "VerifyingAliceBob.sh" in one folder and run the following command in the terminal:
`bash VerifyingAliceBob.sh`
- The verification script is the same one that is run to grade your submissions, only the test vectors are different. Note how the verification script runs the programs in order: first Alice, then Bob, then Alice.
- **You Must submit 2 files for this part of the assignment:**
[alice.c](#) & [bob.c](#)

Common Assignment Notes

- For the correctness of reading from a file and XORing, please use unsigned char.
- For your convenience, we have provided a template file with necessary functions and descriptions: RequiredFunctionsHW1.c
- You may use the functions in RequiredFunctionsHW1.c, modify them, or write your own, modifying the RequiredFunctionsHW1.c file as necessary. **If you modify it, submit your modified RequiredFunctionsHW1.c file alongside your other code files in Canvas.**
- The code that throws errors or does not successfully compile/run on our side, receives zero credit (It doesn't matter if it works on your computer!).
- **Follow the Instructions and all the specific names of output files mentioned above, otherwise there will be 20% deduction for inconvenience.** If you followed the above steps and the output files are correct, you will receive full credit.

Rubric:

- The "Key.txt" file **[5 pt]**
- The "Ciphertext.txt" file **[10 pt]**
- The "Signature.txt" file **[10 pt]**
- The "Response.txt" file **[10 pt]**
- The "Acknowledgment.txt" file **[5 pt]**
- The "A_ctr.txt", "A_nonce.txt", "B_ctr.txt", "B_nonce.txt" files **[5 pt]**